TRABAJO DE APLICACIÓN PROFESIONAL

# Arquitectura del sistema

Anexo 1 — Arquitectura de software, componentes y flujo de facturación electrónica

Sistema de punto de venta, inventario y facturación electrónica
Anexo técnico · Documento generado a partir del código fuente

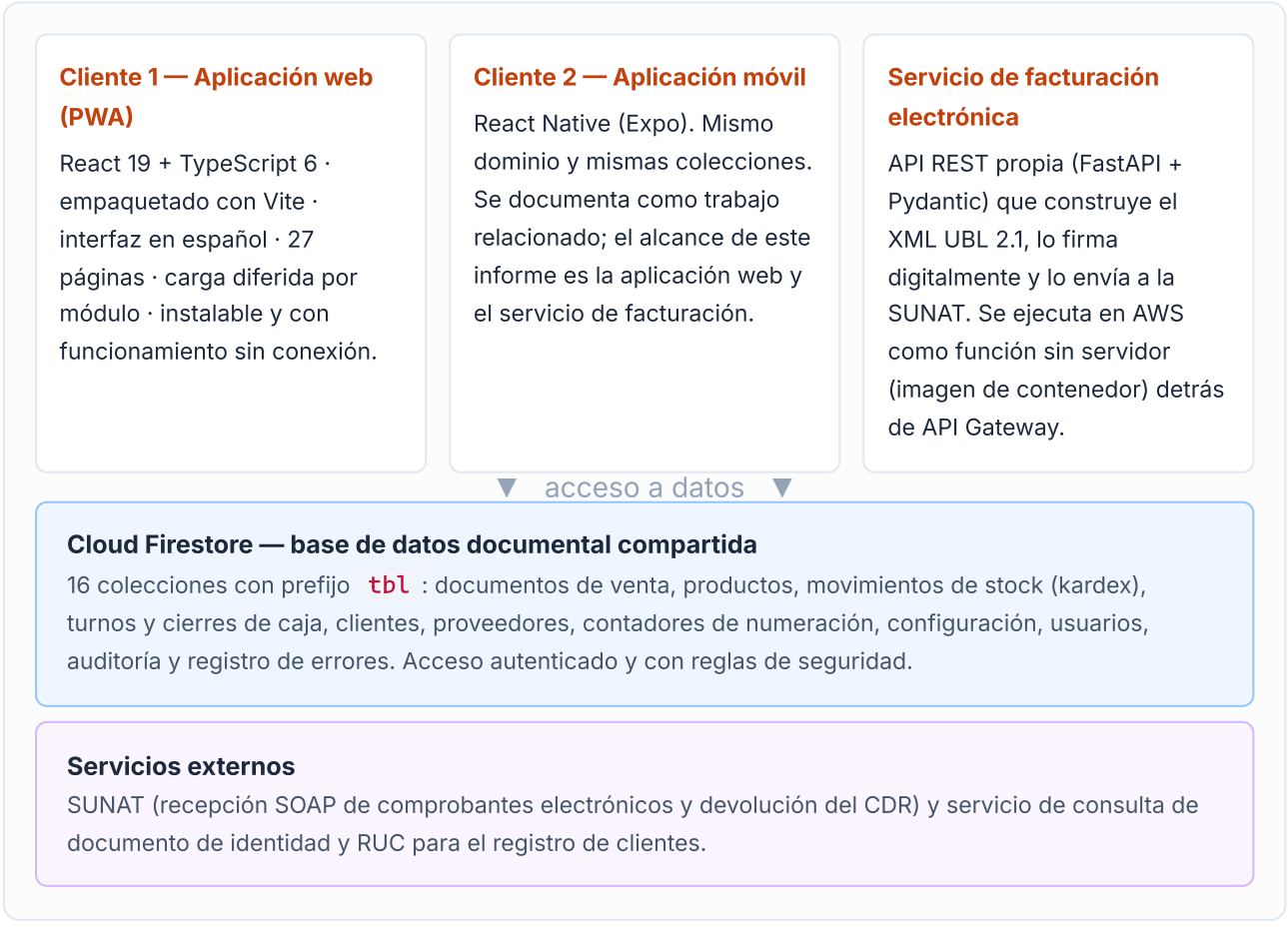
# 1. Introducción

El sistema es una solución de gestión comercial para un negocio de venta al público que integra, en una sola aplicación, el punto de venta, el control de inventario, la administración de caja y turnos, la cartera de clientes y proveedores, los reportes de gestión y la **emisión de comprobantes electrónicos ante la SUNAT**. Está construido sobre una arquitectura de **tres componentes** que se comunican a través de una base de datos documental compartida y de un servicio especializado de facturación:

- **Aplicación web progresiva (SPA)**: es la interfaz principal de operación (punto de venta, inventario, caja, reportes, administración). Se ejecuta en el navegador y se instala como aplicación.
- **Servicio de facturación electrónica**: servicio back-end propio que custodia el certificado digital, construye y firma el comprobante en formato UBL 2.1 y lo envía a la SUNAT. Es un componente separado por una razón de seguridad: *el certificado digital y las claves nunca deben residir en el navegador*.
- **Base de datos documental (Cloud Firestore)**: almacén único de la información operativa (productos, documentos, turnos, clientes). Es la capa que comparten la aplicación web y la aplicación móvil complementaria, de modo que ambas operan sobre los mismos datos.

> **Criterio de diseño general.** Toda la lógica que puede verificarse sin interfaz gráfica ni base de datos se aísla en una capa de módulos puros y se prueba de forma automatizada. Esa decisión es la que permite afirmar que las reglas del negocio (cálculo del IGV, distribución de descuentos, arqueo de caja, numeración de comprobantes) están verificadas por pruebas y no por inspección visual.

# 2. Vista general de componentes



## 3. Arquitectura en capas de la aplicación web

La aplicación web está organizada en tres capas con una regla de dependencia estricta: **cada capa sólo conoce a la que está inmediatamente debajo**. Esto permite probar las reglas del negocio sin navegador y sin base de datos, y evita que la interfaz decida cálculos que deben ser únicos.

**Capa de presentación — `src/pages/` y `src/components/`**
27 páginas y 42 archivos de componentes de interfaz (34 propios y 8 de la librería de interfaz). Maneja el estado de pantalla, los formularios y la navegación (por estado, sin enrutador). Cada página se carga de forma diferida según el módulo que el usuario abre.

▼

**Capa de servicios — `src/services/`**
22 módulos, uno por dominio (ventas, inventario, turnos, notas, consolidación, reportes, usuarios, cajas, compras, proveedores, egresos, kardex…). Es la única capa que habla con Firestore y donde viven las transacciones y los lotes de escritura.

▼

**Capa de reglas del negocio — `src/lib/` (módulos puros)**
29 módulos sin dependencias de Firebase ni de React: cálculo de dinero y redondeo, motor de descuentos y promociones, importes e IGV del comprobante, arqueo de caja y bolsillos de pago, contadores y claves de numeración, armado del comprobante impreso, credibilidad de crédito, reglas de pedidos entre cajero y vendedor, saneamiento de datos antes de escribir. **Cubiertos por 195 pruebas automatizadas.**

**Regla de una sola implementación.** Cuando un cálculo lo usan dos pantallas —o el alta y la anulación de una venta—, vive en `src/lib/`. Ejemplos concretos y verificables: la fórmula del efectivo esperado de un turno (`efectivoEsperadoDe`) estaba escrita a mano en seis lugares, dos de ellos con una variante distinta, y se unificó en una sola función; la lectura del descuento de una venta (`descuentoDeVenta`) y el armado de los importes del comprobante (`itemsParaSunat`) también son funciones únicas, compartidas por la impresión del comprobante y por el envío a la SUNAT. Si lo que se ve y lo que se permite salen del mismo cálculo, no pueden contradecirse.

## 4. Módulos funcionales

| Módulo | Páginas principales | Responsabilidad |
| --- | --- | --- |
| Punto de venta | `Pdv`, `PantallaCliente`, `EscanearCodigo` | Registro de la venta, búsqueda y lectura de productos, presentaciones, listas de precio, descuentos, envío a domicilio, formas de pago y emisión del comprobante. |
| Pedidos y caja | `Pdv` (panel de ventas pendientes), `Turnos`, `ArqueoCaja`, `AuditoriaCaja` | Envío de la venta a caja por parte del vendedor, cobro por el cajero, apertura y cierre de turno, arqueo, conciliación por medio de pago y gastos de caja. |
| Inventario | `Inventario`, `Kardex`, `RotacionInventario` | Catálogo de productos y kits, afectación tributaria por producto, stock mínimo, vencimientos, precios, ajustes y trazabilidad de movimientos. |
| Fiscal y ventas | `Reportes`, `ReporteSire`, `ReporteFechas`, `ReporteTurnos`, `VentasPorEmpleado`, `Analisis` | Notas de venta, consolidación a boleta, notas de crédito y débito, envío a la SUNAT, registro de ventas, reportes por fecha, turno, empleado y categoría. |
| Clientes y proveedores | `Clientes`, `ClienteDetalle`, `Proveedores`, `ProveedorDetalle`, `Compras`, `Egresos` | Cartera de clientes con crédito y saldo, compras que actualizan stock y pagos a proveedores, registro de egresos. |
| Administración | `Configuracion`, `Usuarios`, `UsuarioDetalle`, `Auditoria`, `Dashboard`, `DashboardOperaciones` | Parámetros de la empresa, series y puntos de emisión, listas de precio, plantillas de venta rápida, usuarios con roles y permisos, auditoría de acciones, panel de indicadores. |

## 5. Flujo de emisión de un comprobante electrónico

Es el flujo de mayor criticidad del sistema, porque de él depende el cumplimiento tributario. Se describe en el orden real de ejecución, indicando el componente responsable de cada paso.

1. **Selección del comprobante y cobro (aplicación web).** El cajero elige tipo de comprobante (nota de venta, boleta o factura) y forma de pago. Sólo en este momento se asigna la **serie y el correlativo**, leídos de la colección de contadores; antes del cobro no existe numeración. La venta se registra en una transacción que descuenta el stock, escribe el movimiento de kardex y actualiza el turno.
2. **Construcción de la carga útil (aplicación web).** La función `construirPayloadComprobante` (`src/lib/sunat.ts`) arma el objeto que se enviará al servicio: emisor, receptor, líneas del comprobante y totales. Los importes se calculan **a partir de lo cobrado** —el descuento aplicado se distribuye entre las líneas antes de despejar el IGV— y cada línea se expresa con dos decimales, que es lo que el servicio valida.
3. **Validación de la estructura (servicio de facturación).** El *router* `POST /v1/invoices/emitir` valida la carga útil con modelos Pydantic (`app/schemas/invoice.py`): tipo de documento del receptor, longitud del número de documento, serie de cuatro caracteres, correlativo mayor que cero, cantidades y precios unitarios con dos decimales y totales coherentes. Si algo no cumple, la respuesta indica el motivo y el comprobante no se emite.
4. **Construcción del XML UBL 2.1 (servicio de facturación).** El servicio arma la factura o boleta con la librería `sunat_py` (`build_invoice_xml`) y a continuación aplica un conjunto de ajustes verificables sobre el XML generado: hora de emisión, totales de IGV reconstruidos desde las líneas, **importes exactos** (de modo que base más IGV sea exactamente lo cobrado), correlativos y **punto de emisión** (código de establecimiento anexo, que forma parte del hash del comprobante).
5. **Firma digital.** El XML se firma con el certificado digital del contribuyente (`sign_invoice_xml`). El certificado reside únicamente en el servicio; el navegador nunca lo recibe.
6. **Empaquetado y envío.** El XML firmado se comprime en un archivo ZIP con la nomenclatura exigida (RUC, tipo de documento, serie y correlativo) y se envía por SOAP a la SUNAT.
7. **Respuesta de la SUNAT y constancia.** El servicio interpreta la constancia de recepción (CDR): si el comprobante fue aceptado devuelve el código de respuesta, la descripción y el XML firmado; si fue rechazado, devuelve el motivo. También contempla el código de respuesta que indica que el comprobante ya había sido registrado, para no duplicarlo.
8. **Registro del resultado (aplicación web).** La aplicación guarda en el documento de venta el bloque `sunat` con el estado (aceptado, rechazado, error), el código y la descripción del CDR, el *hash* y el XML y CDR recibidos. Ese estado es el que se consulta después desde el registro de ventas y los reportes fiscales.

| Paso | Componente | Archivo o punto de entrada |
|---|---|---|
| Numeración y cobro | Aplicación web | `src/services/ventas.ts`, `src/lib/contadores.ts` |
| Cálculo de importes y descuentos | Aplicación web | `src/lib/sunat-importes.ts`, `src/lib/descuentos.ts` |
| Armado de la carga útil | Aplicación web | `src/lib/sunat.ts` ( `construirPayloadComprobante` ) |
| Validación y orquestación | Servicio de facturación | `app/routers/invoices.py`, `app/services/invoice_service.py` |
| Estructura del comprobante | Servicio de facturación | `app/schemas/invoice.py` (modelos Pydantic) |
| XML, ajustes y firma | Servicio de facturación | `app/xml_utils.py` , librería `sunat_py` (UBL 2.1) |
| Registro del CDR | Aplicación web | `src/services/sunat.ts` |

## 6. Arquitectura del servicio de facturación electrónica

El servicio sigue el mismo criterio de capas que la aplicación web, con responsabilidades separadas:

**Capa de exposición (API REST) — `app/routers/`**
`POST /v1/invoices/emitir` y `GET /v1/invoices/status` (comprobantes) · `POST /v1/notes/emitir` (notas de crédito y débito) · `GET /v1/ruc` y `GET /v1/dni` (consultas de documento).

▼

**Capa de servicios — `app/services/`**
`invoice_service.py` y `note_service.py` : orquestan la emisión completa (armado, ajustes, firma, empaquetado, envío y respuesta) y la conservación de los archivos generados. `apis_client.py` atiende las consultas de documento de identidad y RUC.

▼

**Capa de integración — `app/xml_utils.py` y librería `sunat_py`**
Generación y ajuste del XML UBL 2.1, firma digital, empaquetado y comunicación SOAP con la SUNAT. `app/sunat_runtime.py` mantiene en memoria —una sola vez por proceso— el certificado y el cliente de servicios web, y `app/config.py` centraliza la configuración leída de variables de entorno.

▼

**Capa de persistencia — `app/storage/firebase_adapter.py`**
Acceso a Firestore desde el servicio cuando corresponde dejar constancia del envío; la configuración y las credenciales se leen de variables de entorno ( `app/config.py` ).

El servicio se empaqueta como contenedor ( `Dockerfile` y `lambda.Dockerfile` ), de modo que se ejecuta como función sin servidor en AWS y tambien puede levantarse en un equipo propio para

desarrollo o pruebas. La dirección del servicio es un parámetro de configuración de la aplicación web, lo que permite apuntar al ambiente de pruebas o al de producción sin recompilar la aplicación.

## 7. Despliegue y operación

| Elemento | Cómo se despliega | Observación técnica |
|---|---|---|
| Aplicación web | Compilación del proyecto y publicación del contenido estático en el servicio de alojamiento de Firebase. | El despliegue se realiza con un comando único; la aplicación se sirve como sitio estático con cabeceras de seguridad. |
| Funcionamiento sin conexión | Service worker y persistencia local de la base de datos. | El service worker conserva hojas de estilo, imágenes y tipografías, aplica estrategia de red primero para las llamadas a datos y avisa al usuario cuando no hay conexión; Firestore mantiene una caché local que permite seguir consultando lo ya cargado y sincroniza las escrituras al recuperar la conexión. ⚠️ El **envío a la SUNAT sí requiere conexión**: si no hay red, el comprobante queda emitido y el envío se reintenta. |
| Servicio de facturación | Funcion sin servidor en AWS (imagen de contenedor) detras de API Gateway; se actualiza publicando una version nueva del servicio. | Un cambio en el servicio no surte efecto hasta reiniciarlo, por eso el procedimiento de actualización se documenta como parte de la operación. |
| Base de datos | Servicio administrado, con reglas de acceso e índices declarados en el repositorio. | Los índices compuestos se declaran en un archivo versionado y se despliegan con el comando correspondiente; un índice creado manualmente fuera del archivo se perdería al forzar la publicación, de ahí la prohibición de usar el modo forzado. |
| Seguridad de contenido | Cabeceras HTTP configuradas en el alojamiento. | Política de seguridad de contenido (orígenes permitidos explícitos), protección contra inclusión en marcos, tipo de contenido declarado y política de referencia. |

## 8. Decisiones de arquitectura y su justificación

| Problema | Decisión adoptada | Justificación |
|---|---|---|
| El certificado digital no puede estar en el navegador | Servicio de facturación separado que custodia el certificado y firma del lado del servidor | Evita exponer credenciales fiscales en el cliente; permite actualizar la lógica tributaria sin volver a publicar la aplicación web |
| La interfaz repetía cálculos críticos | Extraer toda regla a módulos puros ( `src/lib/` ) y probarlos automáticamente | Una sola fuente de verdad por regla; las pruebas se ejecutan sin navegador ni base de datos |
| Comprobantes numerados antes de cobrarse | La numeración se asigna únicamente al cobrar | Evita quemar correlativos y huecos en las series fiscales cuando una venta no se concreta; el pedido sólo tiene un código interno propio |
| Riesgo de descuadre de caja | Una única fórmula del efectivo esperado, usada por todas las pantallas | Antes estaba escrita en seis lugares y dos de ellos calculaban distinto: el cajero podía comparar contra un número diferente al registrado |
| Datos incompletos en las escrituras | Saneamiento obligatorio de los datos antes de escribir en la base | La base documental rechaza valores indefinidos y el compilador no lo detecta: el saneamiento previo convierte un fallo silencioso en un dato consistente |
| Operación con internet inestable | Aplicación web progresiva con caché local y sincronización | El negocio no puede detener la venta por una caída de conexión; el comprobante electrónico se emite al recuperar el enlace |
| Crecimiento a más de un punto de emisión | Las cajas se modelan como puntos de emisión con sus propias series | Permite emitir con la serie correcta por caja y transferir pedidos si una caja deja de operar |
| Necesidad de auditar quién hizo qué | Registro de acciones en una colección de auditoría y registro de errores de la aplicación | Sustenta el control interno del negocio y permite reconstruir incidentes |

## 9. Tecnologías y estándares

| 25 373 | 195 | 16 | UBL 2.1 |
|---|---|---|---|
| LÍNEAS DE CÓDIGO | PRUEBAS AUTOMATIZADAS | COLECCIONES | ESTÁNDAR DEL COMPROBANTE |

| Capa | Tecnología | Uso en el sistema |
|---|---|---|
| Interfaz | React 19, TypeScript 6, Tailwind CSS 3, Vite 8 | Componentes de interfaz, tipado estático estricto, estilos y empaquetado con carga diferida por módulo |
| Datos | Cloud Firestore (Firebase 12), Firebase Authentication | Base documental con transacciones, lotes, persistencia local y autenticación de usuarios |
| Documentos y códigos | `qrcode`, `html5-qrcode`, `recharts` | Representación del código QR del comprobante, lectura de códigos de barras con la cámara y gráficos de los paneles de indicadores |
| Reglas del negocio | Módulos TypeScript puros, runner de pruebas nativo de Node | 195 pruebas automatizadas sobre dinero, descuentos, IGV, arqueo, contadores, pedidos y comprobantes |
| Servicio fiscal | FastAPI, Pydantic, `sunat_py`, UBL 2.1, Docker | Validación de la carga útil, generación y firma del XML, envío SOAP a la SUNAT y lectura del CDR |
| Calidad | TypeScript en modo estricto, Oxlint, pruebas automatizadas | Verificación estática de tipos y estilo, y línea base de advertencias que no debe incrementarse |

## 10. Seguridad y control interno

- **Autenticación** contra el servicio de identidad de Firebase, con persistencia de sesión y **sesión única por dispositivo** para los usuarios operativos: si la misma cuenta inicia sesión en otro equipo, la sesión anterior se cierra.
- **Autorización en la aplicación**: el rol define una plantilla de permisos y cada permiso puede ajustarse por usuario. La interfaz oculta lo que el usuario no puede hacer y los servicios **vuelven a validar las reglas críticas antes de escribir** (por ejemplo, el tope del descuento manual al crear un pedido).
- **Alcance real de las reglas de la base de datos** (verificado en `firestore.rules`): exigen **usuario autenticado** para leer y escribir —y dejan abierta la lectura de la colección de sesiones de la pantalla del cliente—; **no restringen por rol**, decisión documentada en el propio archivo de reglas para no romper flujos de la aplicación móvil. Por lo tanto, quien tenga una cuenta válida puede escribir en la base: **endurecer las reglas por rol es una recomendación de este trabajo, no un control ya implementado**, y así se declara en las limitaciones del informe (Anexo 5).
- **Custodia de credenciales**: el certificado digital, las claves de servicio y los secretos viven en el servicio de facturación y en variables de entorno fuera del control de versiones; nunca viajan al navegador.
- **Integridad de la información**: las operaciones que afectan a varios documentos (venta, stock, kardex y turno; pedido y contador; consolidación y notas de venta) se ejecutan dentro de transacciones o lotes, de modo que no queden estados intermedios.
- **Trazabilidad**: cada acción relevante queda registrada con su responsable, y los errores de la aplicación se registran en una colección propia para su revisión desde el módulo de auditoría.
- **Integridad fiscal**: el comprobante declara los importes efectivamente cobrados, los correlativos no se reutilizan y el resultado del envío a la SUNAT queda almacenado como constancia.

## 11. Relación con los demás anexos

| Anexo | Contenido |
| --- | --- |
| Anexo 2 | Modelo de datos: colecciones, campos, ciclo de vida del documento de venta, contadores e índices. |
| Anexo 3 | Ingeniería de requisitos: actores, requerimientos funcionales y no funcionales, casos de uso e historias de usuario. |
| Anexo 4 | Plan de pruebas: estrategia, inventario de pruebas automatizadas, casos de aceptación, incidencias corregidas y matriz de trazabilidad. |
| Anexo 5 | Informe del trabajo de aplicación profesional por capítulos. |
| Anexo 6 | Diapositivas de sustentación y guion de la defensa. |
| Manual de usuario | `docs/manual-usuario.html` y su versión en PDF: guía de uso del sistema por módulo. |